Resilient parallel computing on volunteer PC grids

Article *in* Concurrency and Computation: Practice and Experience · April 2018

DOI: 10.1002/cpe.4478

4 authors, including:

Jaspal Subhlok
University of Houston
111 PUBLICATIONS **2,299** CITATIONS
SEE PROFILE

Hien Nguyen
Vietnam National Institute of Culture and Arts Studies
7 PUBLICATIONS **32** CITATIONS
SEE PROFILE

Mohammad Tanvir Rahman
University of Houston
4 PUBLICATIONS **16** CITATIONS
SEE PROFILE

Received: 19 April 2017 | Revised: 9 February 2018 | Accepted: 21 February 2018
DOI: 10.1002/cpe.4478

RESEARCH ARTICLE

WILEY

Resilient parallel computing on volunteer PC grids

Jaspal Subhlok | **Hien Nguyen** | **Edgar Gabriel** | **Mohammad Tanvir Rahman**

Department of Computer Science, University of Houston, Houston, TX 77204, USA

Correspondence
Jaspal Subhlok, Department of Computer Science, University of Houston, Houston, TX 77204, USA.
Email: jaspal@uh.edu

Funding information
National Science Foundation, Grant/Award Number: CNS-0834750, MCB-0919974, and CRI-0958464

Summary

Volunteer PC hosts represent massive computation capacity at a low cost but are challenging to employ for general parallel computing. This paper presents the design, execution model, implementation, and evaluation of the *Volpex* framework for robust execution of parallel codes on volunteer PC grids characterized by system and network heterogeneity, varying availability, and frequent failures. The communication model is based on one-sided Put/Get calls to an abstract global shared space enhanced to support multiple autonomous instances of the same process at different stages of execution. Our approach customizes and combines the use of replication, checkpointing, and host selection. This presents formidable challenges that are addressed in this work; efficient checkpointing of distributed replicated processes, dynamic management of redundancy, quick restart in a distributed environment, and application specific host selection. The integrated runtime system is shown to effectively execute moderate size, coarse-grain, communicating codes on a worldwide distributed volunteer environment, a new milestone in volunteer computing. Extensive evaluation is conducted with example scientific codes on a pool of around 600 volunteer hosts. The results demonstrate the trade-offs in deploying checkpointing, redundancy, and host selection, and how these methods combine to provide application performance that is close to the ideal failure free performance.

KEYWORDS

checkpointing, fault tolerance, host selection, parallel execution, replication, tuplespace, volunteer computing

1 | INTRODUCTION

Distributed computing is widely deployed for resource hungry applications that need massive computational power, large storage capacity, high processing speed or quick accessibility of data. Grid computing addresses this challenge with geographically distributed networked loosely coupled clusters. Popular grid computing middleware systems include Globus Toolkit[1] and UNICORE[2,3] (Uniform Interface to COmpute REsources). Ordinary PCs have been employed successfully for large scale scientific computing, most commonly using BOINC[4] as middleware. Web browsers can host middleware for distributed computing, a popular example being Weevilscout.[5]

The BOINC middleware uses volunteered public PCs for scientific applications when idle. It has been remarkably successful, managing as much as 5 Petaflops of aggregate compute power and supporting over 60 scientific research projects. However, this is a tiny fraction of the volunteer compute power that could be exploited. The research presented in this paper aims to enhance the state-of-the-art for parallel computing on volunteer PC grids. We will refer to PCs made available for scientific computing when idle as *volunteer nodes (or hosts, or PCs)* and an execution environment composed of PCs connected by a LAN or Internet as a *volunteer PC grid (VPG)*; even if the PCs are managed hosts in an organization. Volunteer PCs represent a potentially immense but *volatile* resource; they are heterogeneous in terms of architecture, networking, and operating system, prone to failure, and their availability to execute guest scientific applications can change suddenly and frequently based on the PC owner's actions. The state of the art of parallel computing on volunteer nodes is generally limited to applications with “master-slave” or “bag-of-tasks” parallelism. The key goal of the research presented in this paper is to enable execution of communicating parallel codes on volunteer nodes.

Execution of communicating parallel applications on volunteer nodes is extremely challenging because process failures and slowdowns are frequent and the failure or slowdown of a single process impacts the entire application. A fault tolerance approach that incorporates checkpointing and

process redundancy is a necessity.[6] When a checkpoint-restart approach is used, the checkpoints must be taken independently and asynchronously because of the potential overhead of global synchronization. For implementation of checkpoint-restart, as well as process redundancy, the communication data objects must be logged and made available to logically identical processes in case of re-execution in the future; this may be a result of recovery from a checkpoint or a logically identical replica process that is trailing in execution.

The context of this research is the VoLPEx (Parallel Execution on Volunteer nodes) project that has the goal of execution of parallel codes with coarse grain communication on volunteer nodes. The Volpex approach is based on *autonomous redundant processes*, where a process can have multiple concurrent independent replicas that can be created at invocation, re-created from a checkpoint, or terminated, without coordination with other processes or replicas. The overall program progress is dictated by the instance of each process that is furthest ahead in execution. The goal is for the application to achieve seamless forward progress, ie, have the ability to execute the parallel application despite node heterogeneity and routine failures. This paper presents i) Volpex Dataspace API, a programming framework suitable for parallel computing on volunteer nodes and ii) Volpex execution environment to achieve effective application progress in a volunteer environment. This paper is the final report on the Volpex Dataspace project; several aspects of this project have been discussed at various stages in other works.[7-13]

The Volpex Dataspace API is based on anonymous Put/Get style communication among processes pioneered by Linda.[14] This is a logical approach for communication on volunteer nodes as it provides an abstract global shared space that processes can use for information exchange without a temporal or spatial coupling. The Dataspace API can simulate common message passing and shared memory programming styles. The key innovation in Volpex Dataspace API is support for multiple physical Put/Get requests corresponding to a single logical Put/Get request. All requests corresponding to a unique logical request are satisfied with identical data objects. This allows support of implicit (due to checkpoint-restart) or explicit asynchronous redundant execution with the final results guaranteed to be identical to (one of the possible) results with normal execution. Management of execution is orthogonal to this process; the communication infrastructure as well as application processes are unaware whether, and to what extent, redundancy is being employed, or if it is implicit or explicit. The core requirement is to be able to handle asynchronous process replicas in any state of execution.

This paper also presents the Volpex runtime execution environment that achieves the key Volpex objective of continuous forward application progress despite network and host heterogeneity, routine failures, and other challenges of a volunteer environment. A suite of techniques is employed to make this possible: *node selection* to minimize the chance of failure during execution, *redundant processes* to provide failure resistance, *checkpointing* to allow application rollback, and *heartbeat monitoring* and *hot spares* for quick restarts. The runtime system innovations that allow application progress in such a harsh execution environment include: *i)* checkpointing and replication developed and implemented as a unified concept; typically minimal replication is deployed for seamless application progress on failure with checkpoint/restart to maintain a degree of redundancy, *ii)* checkpointing is co-operative; more process replicas imply fewer checkpoints per replica, *iii)* procedures to automatically "identify, kill, release, and replace" healthy processes that are lagging in execution status; this is critical for managing heterogeneity and slowdowns that are not caused by failures, *iv)* a "knob" to manage trade-offs between resource expenditure and performance with control of the degree of replication, checkpointing frequency, and node selection standards, and *v)* a host blacklisting and graylisting mechanism that prevents slowing down of an application by repeated failures, while allowing the vast majority of hosts to work on applications despite occasional transient or application specific failures. The goal of the integrated execution environment is to allow long running applications to execute effectively on volunteer PC grids.

The Volpex execution framework relies on the Dataspace programming framework. The execution environment leverages the Dataspace programming API, which normally stores application communication data objects, for storing checkpoints and other management data objects. The Volpex execution framework is integrated with the BOINC middleware that is used for the basic management of volunteer hosts. Some of the runtime features, in particular node selection, are implemented by modifying the BOINC framework. The challenging task of integration with BOINC was undertaken, first to avoid "reinventing the wheel," but equally importantly, to reach out to the substantial BOINC community of scientists as potential real-life customers for the software modules from this research.

The evaluation of this research is done on a volunteer environment that combines a group of hosts at the University of Houston with other hosts distributed worldwide. The evaluation is driven by a Replica Exchange Molecular Dynamics (REMD) code that is being used actively in physics research, eg, the work of Wang et al.[15] The objectives of the evaluation are i) to demonstrate that Volpex execution framework can effectively manage distributed applications on volunteer hosts and ii) to evaluate the role of node selection, checkpointing, and redundancy in enhancing the performance of long running codes on a volunteer PC grid.

While the primary goal of this research is to transform volunteer PC grids into "virtual clusters" to run a variety of parallel codes, the results from the research can potentially have wider significance. Other emerging platforms for distributed computing, in particular, computational clouds, also have to deal with heterogeneity, errors, and failures, especially as they scale up. The methods developed can play a role in improving reliability of clusters by employing unused cores to run redundant process replicas.

This paper is organized as follows. Section 2 details the properties of volunteer nodes that are critical for the design of programming APIs as well as the execution environment. Section 3 introduces the concept of autonomous redundant processes. Section 4 discusses the Dataspace programming API as the context for this research. Section 5 discusses the design and implementation of the execution environment, including checkpointing, redundancy, and node selection. Section 6 presents evaluation results, and Section 7 discusses related work. Section 8 contains conclusions.

2 | CHARACTERISTICS OF VOLUNTEER HOSTS

We list the characteristics of volunteer computing hosts that present challenges for creating and running communicating parallel applications. In the subsequent sections, we address the design of the Volpex programming model and execution environment to address these challenges.

- *Heterogeneity:* Nodes are heterogeneous in all relevant aspects, such as operating system, CPU model and speed, presence of a graphics processing unit (GPU), memory size, and network bandwidth. They are on the public internet implying that network connectivity to a server and to other nodes changes continuously.
- *Reliability:* Nodes are unreliable and untrustworthy. In addition to catastrophic failures, there can be Byzantine failures where incorrect results are returned. Some nodes have high error or failure rates because of hardware failures (eg, in some cases due to overclocking). In principle, hackers can introduce malicious errors although this is extremely rare in practice.
- *Shared resource:* Most importantly, a volunteer node is not dedicated. Nodes may become unavailable because they are turned off, not allowed to compute due to user activity, or unable to communicate. Periods of unavailability may be short (eg, when the user CPU usage exceeds a threshold and the guest process is disabled) or long (eg, when an office machine is turned off for the weekend). A study of availability in BOINC systems[16] shows that nodes are available about 65% of the time, and about 35% of nodes are available and connected 99% or more of the time. The mean and median availability interval lengths are 20 hours and 8 hours respectively, and about 60% of the volunteer hosts have mean availability interval lengths greater than 4 hours.
- *Accessibility:* Many volunteer hosts are behind firewalls, NATs, and HTTP proxies. Hence, only client initiated communication between a client and a Volpex/BOINC server works consistently. One implication is that scheduling must use a "pull" model in which the server waits for clients to request work. Also, direct communication between hosts may be available but cannot be taken for granted.
- *Resource abundance:* In volunteer computing environments, computational resources are often abundant and the primary challenge is to harness them effectively. In particular, it is generally reasonable to trade-off reduced resource utilization to achieve reliable execution.

3 | AUTONOMOUS REDUNDANT PROCESSES

Our goal is an execution environment that converts a pool of volunteer hosts into a virtual cluster that supports reliable execution of parallel programs. Important considerations are redundancy and checkpointing with minimum coordination in the control layer. We refer to the approach as *autonomous redundant processes* outlined as follows.

- *Redundancy:* A process can have multiple concurrent executing replicas (or instances), typically two but it can be higher. Process replicas may be explicitly created at program invocation or a new instance may be created from a checkpoint to replace a slow or failed process instance, and/or to maintain a certain level of redundancy. Process replicas can also be used to protect the execution against Byzantine failures by using replica results for verification, although this aspect is not explored in this work.
- *Autonomous processes:* Process replicas are not aware of the existence of, or coordinate with, other process replicas and largely act autonomously. A process can checkpoint its state without coordinating with other processes. A process can be recreated from a checkpoint irrespective of whether the original process is dead or alive, and without coordination with other replicas or other processes.

The execution model ensures consistent and seamless application progress while at least one replica of each process is active; other replicas may be dead or lagging. This is illustrated in Figure 1. Lagging replicas may be used for result verification and for providing protection from Byzantine faults due to software/hardware errors or malicious hosts. Redundant computations may appear "wasteful" but are important and complement checkpointing for a host of challenges; in particular, data corruption, malicious software, nodes and networks of varying speeds, transient delays, and frequent failures.

Figure 2 shows the performance impact of replication and checkpointing based on a simple theoretical model.[13] According to the model, the normalized expected performance overhead can be represented by

$$NormalizedOverhead = \frac{1}{\left(1-(1-e^{-\lambda \cdot T_c})^r\right)^n} + \frac{T_s}{T_c},$$

where T_c is the checkpoint interval, T_s is the time to save a checkpoint, n is the number of processes, r is the number of replicas for each process, and $e^{-\lambda \cdot T_c}$ is the success probability distribution (assuming an exponential distribution) of a single process instance to reach a checkpoint without failure. *NormalizedOverhead* is the ratio of completion time under the given scenario versus execution time with fault free execution on similar nodes. For Figure 2, n is set to 64, and T_s is set to 120 seconds.

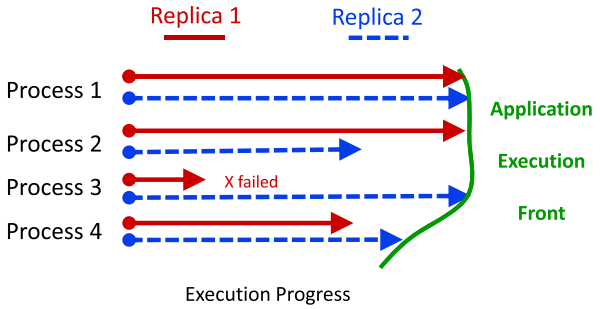

FIGURE 1 Application progress is determined by the leading front created by the fastest replica for each process

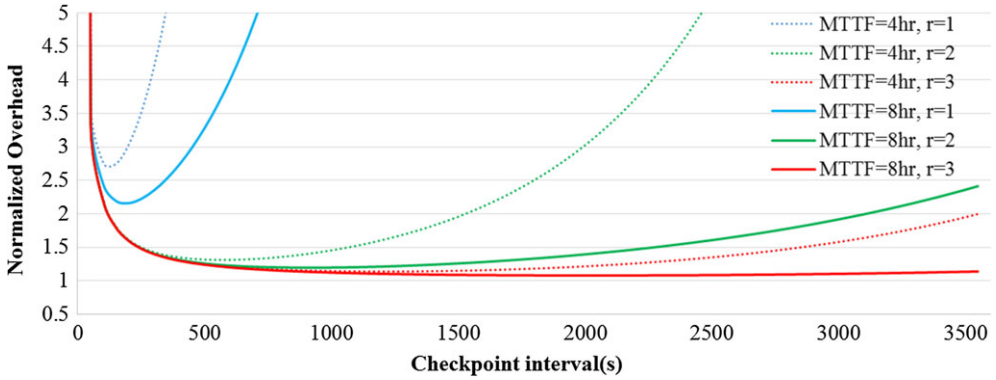

FIGURE 2 Normalized overhead for execution on volunteer nodes with different checkpoint sizes and degrees of replication

4 | DATASPACE COMMUNICATION API

The Volpex Dataspace communication library consists of asynchronous calls to add, read and remove data objects to/from an abstract shared *Dataspace*, with each object identified by a unique *tag*, which is a global index into the Dataspace. The concept of a Dataspace is similar to that of a tuplespace in Linda[14] and many variants. The main communication calls are abbreviated as follows:

Volpex_put(tag, data); A *Volpex_put* call writes the data object *data* into the abstract Dataspace identified with a *tag*. Any existing data object with the same tag is overwritten.

Volpex_read(tag); A *Volpex_read* call returns the data object that matches the *tag* in the Dataspace.

Volpex_get(tag); A *Volpex_get* call returns the data object that matches the *tag* in the Dataspace, and then removes that data object from the Dataspace.

Volpex_read and *Volpex_get* calls are identical except that *Volpex_get* also clears the matched data object from the Dataspace. Both *Volpex_read* and *Volpex_get* are blocking calls: If there is no matching data object in the Dataspace, the calls block until a matching data object is added to the Dataspace. A *Volpex_put* call only blocks until the operation is completed. Additional calls are available in the API to retrieve the process Id and the number of processes, and to initialize and terminate communication with the Dataspace server. The basic API is outlined in Table 1.

Dataspace Execution Model The basic semantics of the communication operations are straightforward as listed with the API above. However, managing autonomous redundant processes, implying that multiple copies of the same process may execute asynchronously, is the main challenge.

TABLE 1 Volpex Dataspace communication API

int volpex_put (const char tag, int tagSize, const void* data, int dataSize)*
int volpex_get (const char tag, int tagSize, void* data, int dataSize)*
int volpex_read (const char tag, int tagSize, void* data, int dataSize)*
int volpex_getProcId (void)
int volpex_getNumProc(void)
int volpex_init(int argc, char argv[])*
void volpex_finalize(void)

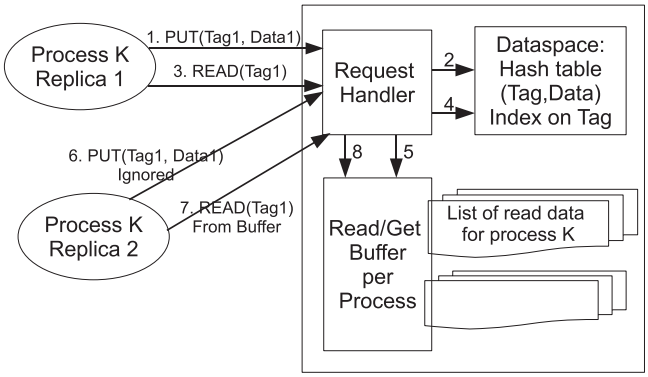

FIGURE 3 Volpex Dataspace server design

The key problem is that a logical call (with side effects) may be executed repeatedly at different times. To ensure consistency, the execution model consists of the following fundamental rules:

1. *Atomicity rule:* The basic *put/read/get* operations are atomic and executed in some global serial order.
2. *Single put rule:* When multiple replicas of a process issue a *Volpex_put*, the first writer accomplishes a successful operation. Subsequent corresponding *Volpex_put* operations are ignored.
3. *Identical get rule:* The first replica issuing a *Volpex_get* or a *Volpex_read* receives the relevant value stored at the time in the Dataspace. Subsequently, replicas of the corresponding *Volpex_get* or *Volpex_read* receive an identical value, independent of the state of the Dataspace objects at the time they are executed.

If these rules are followed, all process instances are guaranteed to execute identically.* Application results are consistent for deterministic as well as nondeterministic parallel programs as long as the sequential execution of a single process code is deterministic.

Dataspace Implementation The communication API is implemented with the client server paradigm. The processes connect to a Dataspace server that services communication requests. The main challenge is in implementing the execution model with support for redundant processes. The implementation of the Volpex Dataspace API must conform to the execution semantics discussed above. The atomicity rule is satisfied by a server that processes only one client request at a time. In order to satisfy the *single put* and *identical get* rules of the execution model, additional machinery is needed. Each logical communication call (*put*, *get*, or *read*) is uniquely identified by the pair: *(process_id*, *request_number)*, where the *process_id* is the volpex Id of a process as returned by *volpex_getProcId()*, and *request_number* is the current count in the sequence of requests from a process. When a communication call is issued by a process, the *process_id* and *request_number* are appended to the message sent to the Dataspace server to service the request. For replicated calls corresponding to the same logical call, the (*process_id*, *request_number*) pairs are identical. This allows the identification of a new call and subsequent replicated calls.

The server implementation maintains the current *request_number* for each process, which is the highest request number served for that process so far. The server also maintains two logically different pools of storage, as shown in Figure 3.

- *Dataspace table:* This storage consists of the logically "current" data objects indexed with tags.
- *Read log buffer:* This storage consists of data objects recently delivered from the Dataspace server to processes in response to *get* and *read* calls. Each object is uniquely identified by *(process_id*, *request_number)*.

When a communication API call is executed by a process, a message is sent to the Dataspace server consisting of the type and parameters of the call and *(process_id*, *request_number)* information. A request handler at the server services the call as follows:

- *Put:* If the request number of the call is greater than the current request number for the process (a new put), the data objected indexed with the tag is added to the Dataspace table. If the request number of the call is less than or equal to the current request number (a replica put for which the data object must already exist on the server), no action is taken.
- *Get or Read:* If the request number of the call is greater than the current request number for the process (a new get), then i) the data object matching the tag is returned from the Dataspace storage and ii) a copy of the data object is placed in the read log buffer indexed with *(process_id*, *request_number)*. Additionally if the call is a *get*, the data object is deleted from the Dataspace table (but retained in the read log buffer).

  If the request number of the call is less than or equal to the current request number (a replica get for which the data object must exist in the read log), the data object matching *(process_id*, *request_number)* is returned from the read log buffer.

*There is an implicit assumption that floating point arithmetic works identically on all nodes.

The design of the Dataspace server is illustrated in Figure 3.

The description in this section is a basic outline of the Dataspace server implementation. We now briefly comment on how some of the challenges are addressed. The Dataspace server appears to be a single point of failure and a potential bottleneck. However, in our current implementation, the server employs multiple threads for sending and receiving data, which mitigates the problems associated with a potential data transfer bottleneck. It should also be noted that the server is normally hosted on reliable dedicated hardware. Distributed and fault tolerant implementations of a Dataspace server are a subject of ongoing research. The current implementation does not specifically account for corrupted data or malicious hosts. However, the *Put* implementation can be modified to require multiple identical puts for reliability, or use non-conforming *Puts* to identify malicious hosts. Finally, the size of the storage buffers can theoretically grow indefinitely during program execution. This is being addressed by employing secondary storage on demand and other practical approaches to storage management.[12]

5 | EXECUTION ENVIRONMENT

The execution environment is critical for parallel Volpex jobs to run effectively on volunteer hosts characterized by heterogeneity, fluctuating availability, failures, and limited accessibility. We first describe the basic functions and implementation of the execution framework, and then discuss the key features in more detail.

5.1 | Execution manager overview

The basic tasks of the execution manager are as follows.

1. *Initiation:* A new Volpex job provides the execution manager with the number of worker processes and the list of application versions available (eg, Windows, Linux, GPU types, etc.). Several additional parameters may be provided, including total floating point operations (FLOPS) reflecting nominal execution time, FLOPS between checkpoints, size of checkpoints, memory and disk requirements, and communication bandwidth. The execution manager starts recruiting hosts for the job by selectively accepting volunteer hosts that contact the server. A set of additional hosts are recruited as *hot spares*. When sufficient hosts are recruited, application execution starts simultaneously on all selected processing nodes. In our applications, processes often communicate among themselves via the Dataspace server. If application execution starts without recruiting sufficient nodes, it may get delayed as recruitment may be necessary during execution. Processes may have two or more replicas at initiation based on application request and execution manager policy.
2. *Progress:* During execution, the hosts create and store independent checkpoints. Active hosts send routine "heartbeat" messages that allow the execution manager to monitor hosts. When a replica dies or slows down so much that it becomes irrelevant to the progress of the application, additional process replicas are created by activating hot spares, typically from a checkpoint. Lagging live replicas are shut down. This process is detailed later in this section.

Figure 4 outlines the implementation of the execution management framework. The BOINC middleware is used for core execution management tasks such as dispatching binaries and data files to hosts and verifying and collecting results. The host selection is implemented as an extension to

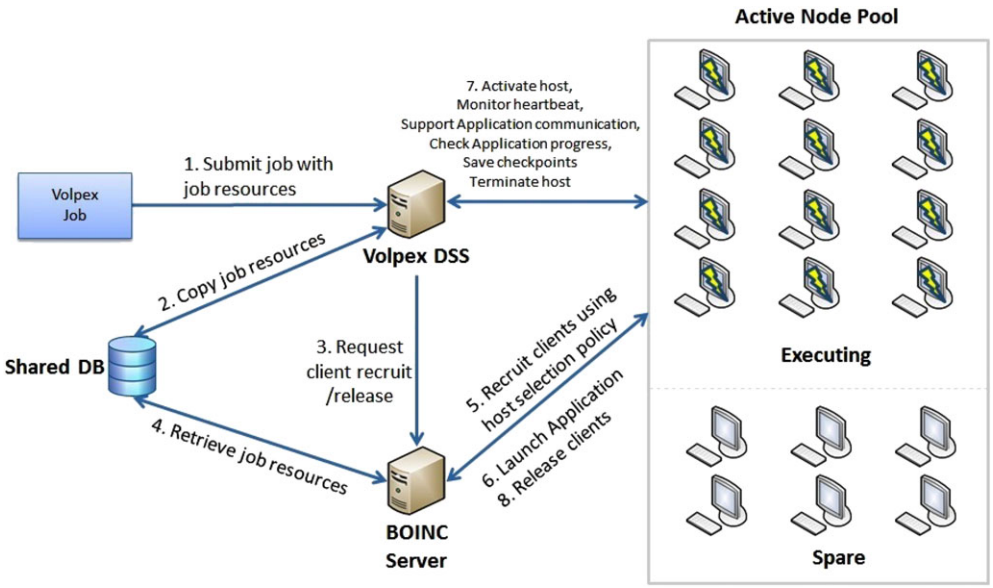

FIGURE 4 System Execution Framework

the core BOINC scheduler. The Dataspace communication API is used by the hosts for sending heartbeats and checkpoints, and by the execution manager to activate or deactivate hosts. A shared database allows access to information across BOINC and Volpex Dataspace server boundaries. The actions taken by the execution manager in the course of execution of a Volpex job are summarized in Algorithm 1.

Algorithm 1 Execution Manager Workflow

```
1:  procedure EXECUTIONMANAGER
2:      JobDB ← Accept Job
3:      do
4:          ActiveNodePool ← Recruit nodes
5:          SpareNodePool ← Recruit nodes
6:      while not enough host
7:      Launch Application
8:      do
9:          switch event do
10:             case CheckpointRequest
11:                 if Enough progress since last checkpoint then
12:                     DSS ← SaveCheckpoint.
13:                 if Process lagging too much then
14:                     Kill process.
15:             case HostFailure
16:                 if Spare node available then
17:                     Select node from SpareNodePool
18:                     Copy latest Checkpoint from DSS and start process
19:                 if No Spare node available then
20:                     SpareNodePool ← Recruit nodes
21:                     Select node from SpareNodePool
22:                     Copy latest Checkpoint from DSS and start process
23:             case ProcessCommunicationRequest
24:                 if Request for PUT operation then
25:                     DSS ← PUT data
26:                 if Request for GET operation then
27:                     GET data from DSS
28:     while Application not complete
29:     Release Recruited nodes
```

5.2 | Host selection

For efficient execution on volunteer nodes, it is important that only hosts that have acceptable execution attributes be allowed to participate in the execution of an application. This is especially important for execution of communicating parallel codes as the slowdown or failure of a single process affects the entire execution. The objective is to *maximize the minimum performance* rather than maximizing the average performance. Host selection is primarily based on host characteristics. However, the specific history of execution of a particular application on specific hosts is also relevant for long running applications. We separately discuss these two aspects and then present the overall mechanism of host selection.

Host characteristics We list the important characteristics of hosts that are considered for host selection.

- *Computation and storage capacity:* Only hosts above a minimum threshold of CPU capacity, memory storage, available communication bandwidth, and available disk capacity are selected. This reduces the probability that a single host becomes a bottleneck because of limited system or communication capacity. The specific cutoffs are part of the system configuration that can be adapted for specific applications.
- *Expected future host availability:* When a host fails or becomes unavailable, the execution may have to be restarted from the beginning, or from a previous checkpoint, while the rest of the hosts are blocked for communication. Considerable research has reported on the predictability of host availability in volunteer environments.[17-19] For the evaluation in this work, we employ a basic predictor called *Lastval*, which is simply based on the availability of the host in the immediate past. This predictor, although very simple, has been shown to be competitive with the best predictors in simulations.[18]

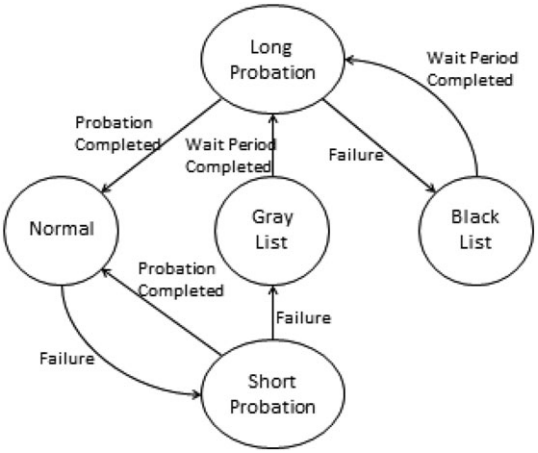

FIGURE 5 Black/Gray Listing Policy

Blacklisting policy for hosts Process failure is routine in volunteer computing but the reasons and characteristics of failures vary. A failure may happen only for a specific application on a particular host architecture. A unique aspect of volunteer computing is that an application can be executing on a wide range of architectures, operating systems, and software configuration. Hence, it is virtually impossible for the application developer to fully test an application in every possible configuration. It is intuitively clear that if an application process fails because of reasons specific to a host, such as software incompatibility or misconfiguration, it is highly likely that another process of the same application will also fail, but a process from another application may not be affected. However, it is not possible to determine if a process failure is caused by an application specific factor, or other reasons such as a host or network failure. System failures may also be transient that are fixed, eg, with a system reboot, or a medium term that are fixed, eg, with a software upgrade. The reason for failure may also be long term or permanent, such as a hardware malfunction. *The challenge is to prevent a very small number of hosts from repeatedly failing and slowing down an application, while allowing the vast majority of hosts to work on the application despite occasional failures.*

The approach taken to address this problem is illustrated in Figures 5 and described as follows. When a process fails on a host, the host is put into the *Short probation mode*. If no failures occur during this probation period, the host returns to normal execution. However, in case of a failure, the host is added to a *Gray list*. When a host is on the gray list, it is not allowed to execute the application for a significant period of time. After that it starts a *Long probation period*. If the long probation is completed without failures, the host proceeds to normal execution. However, in case of a failure, the host is added to a *Black list*. The host waits for a long period of time on a black list, and then starts a long probation period; subsequently the host transitions back to the black list in case of failure, and normal execution if the probation period is completed without any failure.

For the experiments presented in this paper *Short probation period* is set to 24 hours, *Long probation period* is set to 48 hours, waiting period on the *Gray list* is set to 48 hours, and waiting period on the *Black list* is set to 1 week. Note that no host is banned permanently from receiving work.

Host selection Mechanism Host selection has been implemented by modifying the basic BOINC scheduling policy.[20] When an application request is obtained, the runtime system starts recruiting hosts as they contact the application server, subject to the host selection policy based on the application specific and application independent host characteristics discussed earlier. Sufficient hosts are recruited to start application execution and to be available as *hot spares*. Note that volunteer computing with BOINC employs a "pull" based policy, where hosts initiate all contact with the server. Hot spares are recruited ahead of time so that (i) execution does not block when another host is needed and the BOINC server has to wait until it is contacted by an appropriate host and ii) to avoid the startup time of shipping executables etc. once a replacement host has to be activated. After a host is recruited, it is provided with the necessary binaries and data files. The host subsequently contacts the execution manager with a Dataspace API call, to request a process ID. Once sufficient hosts are recruited, execution is initiated on all hosts, by providing them with process IDs. Hot spares are denied a process ID, until they are needed. Depending on the user preferences and policy parameters, one, two, or more instances of each process may be initiated. When the number of hot spares dips below a preset criteria, a fresh set of hosts are recruited until sufficient hot spares are available.

5.3 | Checkpoint and restart

Volpex design is based on integrated redundancy and checkpoint management. The challenges for checkpoint management in the Volpex execution environment stem from the following considerations: i) Volpex processes and process replicas are distributed and may be in different stages of execution, ii) there can be a varying number of process replicas, and iii) checkpointing overhead has to be managed even though an individual process

that issues a checkpoint does not have the information necessary to make optimal checkpointing decisions. We discuss how these are addressed in the Volpex execution environment.

Currently, only application level checkpointing is supported, although system level checkpointing (eg, the work of Duell et al[21]) may be supported in the future. Uncoordinated checkpointing[22] is the only viable option in a volunteer environment as coordinated checkpointing[23] requires synchronization of all processes. Global synchronization to create a checkpoint is especially impractical when multiple replicas of a process exist and the objective is for the application execution front to move forward with the leading replica and to never have to wait for lagging replicas. Recovery from an uncoordinated checkpoint requires that re-executed communication calls from past logical execution states must also receive a logically correct response. However, this is a central feature of the Dataspace communication API designed to support redundancy and process restart from a checkpoint.

Volpex checkpoints are stored on the Dataspace server via a Put(DataObject) call. While the application issues the calls to create checkpoints, the execution manager controls the maximum frequency at which a checkpoint is generated and stored, in order to prevent the system from being overwhelmed by excessive checkpoints. Essentially, the execution manager ignores an application call to checkpoint if sufficient time has not elapsed since the previous checkpoint.

Recording of checkpointing is also co-operative between the replicas of a process. A new checkpoint is created and stored only when it represents meaningful application progress beyond the most recent recorded checkpoint across replicas. As replicated processes may be at different stages of execution, it is inevitable that some process replicas will attempt to checkpoint their state after another replica has already recorded a checkpoint that represents a state further ahead in logical execution. A checkpoint that represents a state older than the state represented by the most recent checkpoint in the system is of no value. Such a checkpoint request is identified as an obsolete request with a logical timestamp mechanism and the request to checkpoint is ignored. An important benefit of such co-operative checkpointing is that the checkpoint frequency for a process instance decreases as the number of replicas increase. Also, lagging process replicas will not need to checkpoint at all.

Finally, a restart is achieved by activating a spare node with the most recent checkpoint of the process that needs to be restarted.

5.4 | Runtime process control

Once execution is in progress, an important responsibility of the Volpex execution environment is to generate new process instances when needed and terminate process instances that are not useful for application progress. The default policy followed by the Volpex execution environment is to maintain the degree of redundancy with which an application was started. A new process instance is created when an existing process instance is *dead* or *obsolete*. We first discuss how dead and obsolete process instances are identified.

Dead processes Every process periodically sends a *heartbeat* to the Dataspace server. If several heartbeats are missed, ie, no heartbeat is received for a preset period of time, then the process is considered *dead*. The reason may be that the host had a failure, the host process was terminated due to user activity, or there was a network outage.

Obsolete processes A host may be providing heartbeats, but for a variety of reasons, it may be making very slow progress. As a result, it is possible that a process replica gets so far behind in execution that there is no benefit to the application in keeping it running. In this case, the process replica is considered obsolete. Obsolete processes are detected by a logical timestamp reported with every checkpoint request that is tracked by the execution manager. For example, suppose a lagging process replica L makes a checkpoint request (say for checkpoint25) at logical time T_L and the checkpoint request for checkpoint25 by the fastest replica of the process was in T_F. If $(T_L - T_F) >$ LaggingThreshold then replica L is marked as obsolete.

Whenever a process instance is considered dead or obsolete, it is terminated and a spare host activated to take its place. A process instance is terminated by sending a *Kill* response to a heartbeat. Note that a dead process may appear to become alive (a zombie) and send a heartbeat in the future if the termination was not completed successfully. In such instances, a fresh attempt is made to terminate the process instance. Terminating an obsolete process allows the host to be used for another application.

The creation of a process instance is simply done by providing the process ID to one of the spare hosts. After receiving a process ID, a new process automatically checks for the most recent checkpoint in the system for that process. If a checkpoint is found, execution starts from that state, else execution starts from the beginning.

6 | USAGE AND RESULTS

The Volpex Dataspace programming framework has been implemented, deployed, and tested with several benchmarks and applications. The execution management system is integrated with the Volpex Dataspace system and the BOINC scheduler. Experimentation and validation was done on a volunteer host pool that consists of ordinary internet-connected desktops, as well as compute clusters and lab computers. The experiments were repeated over several months as the execution time varies in volunteer computing. The results presented in this paper focus on evaluating the

performance of the Dataspace server and validating the execution environment, rather than present the overall usage of Volpex. The following are the specific objectives:

1. Demonstrate that the Volpex execution environment integrated with the Dataspace API and BOINC scheduler effectively converts a volunteer node pool to a virtual cluster capable of executing long running parallel jobs.
2. Demonstrate that the Dataspace API operations are executed efficiently.
3. Evaluate the importance of the execution management mechanisms, specifically node selection, replication, and checkpointing, on the performance of parallel programs executed on a volunteer environment.

6.1 | Evaluation testbed

For the results presented in this paper, the codes were executed on a volunteer environment managed by BOINC middleware consisting of cluster nodes from University of Houston and a substantial number of volunteer nodes from around the world. The cluster nodes inside University of Houston are in active use for other tasks but available to BOINC/Volpex when idle. The configuration of a cluster node was a 2.4-2.8 GHz Intel Xeon processor with 2 GB of memory and running an Unix operating system. Our scheduling policy allows multiple Volpex processes to run on a node at a given point in time as long as they are executed on separate processing cores. We verified that virtually all our experiments employed a mix of on-campus nodes and volunteer nodes from around the world. Approximately, 20% of the nodes were on the university campus and the remaining 80% were distributed worldwide.

The Dataspace server and the BOINC server were running on an AMD Athlon 2.4 GHz dual core machine with 2 GB memory running Ubuntu 9.01. The server and on-campus client nodes were on 100 Mbps subnets that are part of the UH campus LAN consisting of 100 Mbps and 1 Gbps links.

Several programs have been developed for the Volpex environment including latency and bandwidth microbenchmarks, a *Sieve of Eratosthenes (SoE)* program, a *Parallel Sorting by Regular Sampling* program, and a simple implementation of the *Map-Reduce* framework. We have also adapted a *Replica Exchange Molecular Dynamics (REMD)* code from an active physics project to run under Volpex. For this paper, we report results only from the latency and bandwidth microbenchmarks, Sieve and REMD programs.

6.2 | Benchmarking of API calls

In the first set of experiments, we recorded the time taken to execute the different API calls with varying message sizes and varying degrees of replication. We ran the experiments with one cluster node and one on-campus node with BOINC client installed as a volunteer node. The effective bandwidth delivered by the server in response to *put* operations from a cluster node is presented in Figure 6A. Note that the bandwidth presented is the aggregate bandwidth delivered by the server in response to all clients in case of replication. While the relative overhead observed could be slightly higher on a Gigabit Ethernet or faster network, Fast Ethernet represents a realistic network for volunteer environments, where the network between clients and the Volpex server can range from a few hundred kb/s for home PCs to Gigabits in campus settings.

We observe that the general bandwidth trend is typical of a 100 Mbps LAN environment. The effective bandwidth increases with the size of the data object but flattens out around 12 Mbps (or 96 Mbps), which is just below the network capacity of 100 Mbps. Hence, the system overhead is not significant. The figure also shows that the effective bandwidth with 2 and 4 replicated processes is virtually identical to that without replication except for a very slight reduction in the midrange of message sizes. It is instructive to recall how replicated *put* operations work. The first *put* actually transfers the data object over the network, and replica *put* calls are returned without any data transfer. Thus, the total network traffic does not increase significantly with replication. Hence, it is not surprising that the effective bandwidth delivered by the server is not significantly affected. The slight reduction is attributed to the overhead of processing of *put* calls from other replicas.

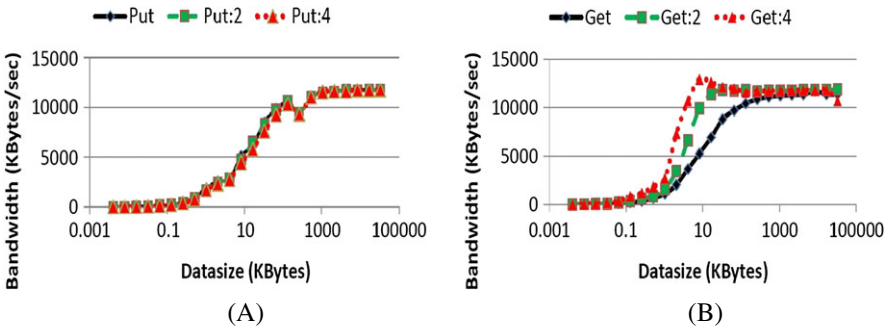

FIGURE 6 Aggregate Bandwidth for PUT and GET operations with and without replicas on a cluster. A, PUTs; B, GETs

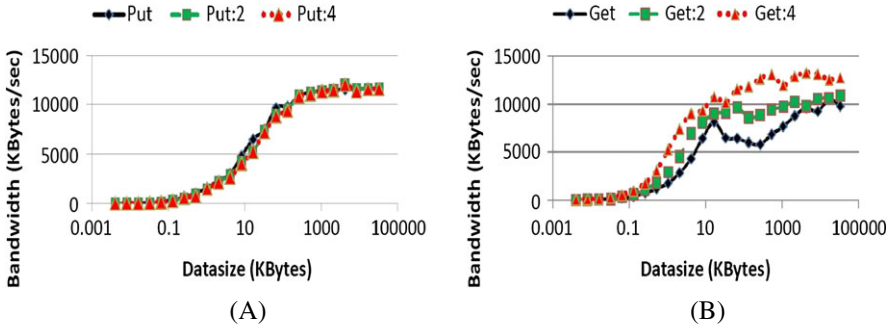

FIGURE 7 Aggregate Bandwidth for PUT and GET operations with and without replicas on a Volunteer environment. A, PUTs; B, GETs

The results for the delivered bandwidth for *get* operations on a cluster are presented in Figure 6B. We omit the results for *read* operation as they are virtually identical to those for the *get* operation. Without replication, the performance of *get* operations is very similar to the performance of *put* operations and the same discussion applies. However, the behavior with replicas is very different. It is instructive to recall that replicated *get* operations are handled very differently from *put* operations. Each replicated *get* call leads to the entire data object being transferred from the server to a client replica. Hence, for a degree of replication of *k* the network traffic for *get* calls increases by a degree of *k* while it remains unchanged for *put* operations.

Figure 6B shows that the aggregate bandwidth delivered by the server *increases* significantly with replication except for very high message sizes where the bandwidth is limited by the network capacity. The server is able to register a higher bandwidth as 2 and 4 replicas imply that the aggregate rate at which the data is being demanded by the clients increases by a factor of 2 and 4, respectively.

Figure 7 presents results from the evaluation of performance of *put* and *get* operations on volunteer environment nodes. The results are virtually identical for *put* operations and slightly lower on average and less predictable for get operations. The fact that the results are not significantly different for cluster and volunteer nodes is not surprising since the Dataspace server and the 100 Mbps LAN through which the clients and the Dataspace server are connected are identical. The key difference that the cluster hosts are dedicated and faster is not a major factor in this scenario.

In summary, the *put* and *get* operations are able to utilize the bulk of the available network bandwidth under moderate data size objects. The implication is that there are no fundamental inefficiencies in the design or the implementation of the Dataspace server, at least for routine operations.

6.3 | Evaluation of execution scenarios

A set of experiments was conducted to evaluate the practical benefits of execution parameters; specifically degree of replication, checkpoint intervals, and node selection. We briefly describe the *Sieve of Eratosthenes* (SoE) code and *Replica Exchange Molecular Dynamics* (REMD) codes, and then present a suite of results employing them.

Sieve of Eratosthenes (SoE) is a well known algorithm for finding prime numbers. In the parallel SoE implementation on Volpex, one process repeatedly broadcasts blocks of prime numbers through the Dataspace API for use by other processes for their own computations. The communication time in SoE can be a significant fraction of the overall processing time. For the results presented, primes up to 2 billion were computed.

Replica Exchange for Molecular Dynamics (REMD) is a real world application used in protein folding research.[24] Each node runs a piece of molecular simulation at a different temperature using the AMBER program.[25] At certain time steps, temperature data is exchanged between neighboring nodes based on the Metropolis criterion, in case a given parameter is less than or equal to zero. A snapshot of such exchange of temperatures is presented in Figure 8, from one of the experiments. It shows how the temperatures are exchanged for an execution with 10 temperature replicas and 5 steps. In each step, temperatures that are swapped are highlighted with the same background pattern.

# Step	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10
1	290	300	310	320	330	340	350	360	370	380
2	290	300	310	320	340	330	360	350	380	370
3	290	310	300	340	320	360	330	350	380	370
4	310	290	340	300	360	320	350	330	370	380
5	310	340	290	360	300	350	320	370	330	380

FIGURE 8 REMD Temperature(K) swaps: 10 temperature replicas for 5 steps

In our implementation of this code, the Dataspace API is used for i) storing process-temperature mapping, ii) synchronization of the processes at the end of each step, iii) identification and retrieval of energy values from neighboring processes, and iv) swapping of temperatures between processes when needed. In the REMD implementation, all processes repeatedly perform a computation step and exchange results. While the processes in REMD communicate regularly, the volume and frequency of the communication is relatively low and the computation to communication ratio is high. For the basic REMD data set and parameters employed for evaluation, each execution step on typical volunteer nodes takes 30 seconds, which is followed by a replica exchange communication step, where each process exchanges 50 kB of data. Hence, the processing time is dominated by computation, and possibly synchronization between processes. The actual communication time is a small fraction of computation time on almost any network. Both SoE and REMD represent common patterns of communicating parallel codes that are suitable for a volunteer environment.

The goal of the experiments was to evaluate the impact of runtime settings in Volpex for common execution patterns. Both SoE and REMD codes were executed for 64 processes with approximately 600 volunteer nodes deployed for execution. The approximate execution times for the SoE and REMD codes for the datasets employed for evaluation are approximately 100 seconds and 30 minutes, respectively, on good nodes without failures. A"good" node is a typical cluster node at the time of the experiments, which would be similar to a midrange desktop system purchased around 2012. The execution scenarios used for evaluation were combinations of the following runtime environment parameters:

- 1, 2, 3, or 6 replicas per process.
- With or without node selection.
- With checkpoint interval of 15 minutes or 1 hour.
- With a checkpoint size of 50 kB or 5 MB.

Checkpointing was employed only for the REMD code as the typical runtime of SoE is too short for checkpointing to be beneficial in most instances. The checkpoint intervals are set based on programmer's intuition. A better understanding of the optimal checkpoint interval can be found in the work of Rahman et al.[13] Also, the size of the checkpoint was synthetically increased to 5 MB for a suite of REMD experiments to evaluate the impact of a large checkpoint size. The run-times of different experiments with the same scenario vary a lot based on the allocated node configuration and network congestion. That is why, the codes were run 20 times in each scenario. The results are presented as quartile box plots with the mean of the data set noted along with the median and quartiles. As expected for a volunteer environment, the results often vary dramatically from run to run.

Impact of replication Figure 9 presents the first set of experimental results for the REMD and SoE codes. We explain the terminology used in labeling of the scenarios along the *x axis* in the plot for REMD that is used for all graphs in this section. The boxes are labeled *1Rep, 2Rep, 3Rep, and 6Rep* to indicate the degree of replication employed. The label *0Sel* is used to indicate that no host selection was employed; label *1Sel* is used in later plots to indicate application of host selection. Label *CP15* denotes a checkpoint interval of 15 minutes, and *CS50K* denotes a checkpoint size of 50 kB; other values for checkpoint interval and checkpoint size are used in later plots. Finally, the box plot for each scenario shows the *Max, Min, Median, 25th quartile*, and 75th *quartile* of execution time. The *Mean* is also marked in the graph.

Figure 9 presents experimental results for REMD and SoE executed with replication levels of 1,2,3, and 6. No host selection was employed. For REMD, fixed checkpoint interval of 15 minutes and checkpoint size of 50 kB were used. No checkpointing was used for SoE.

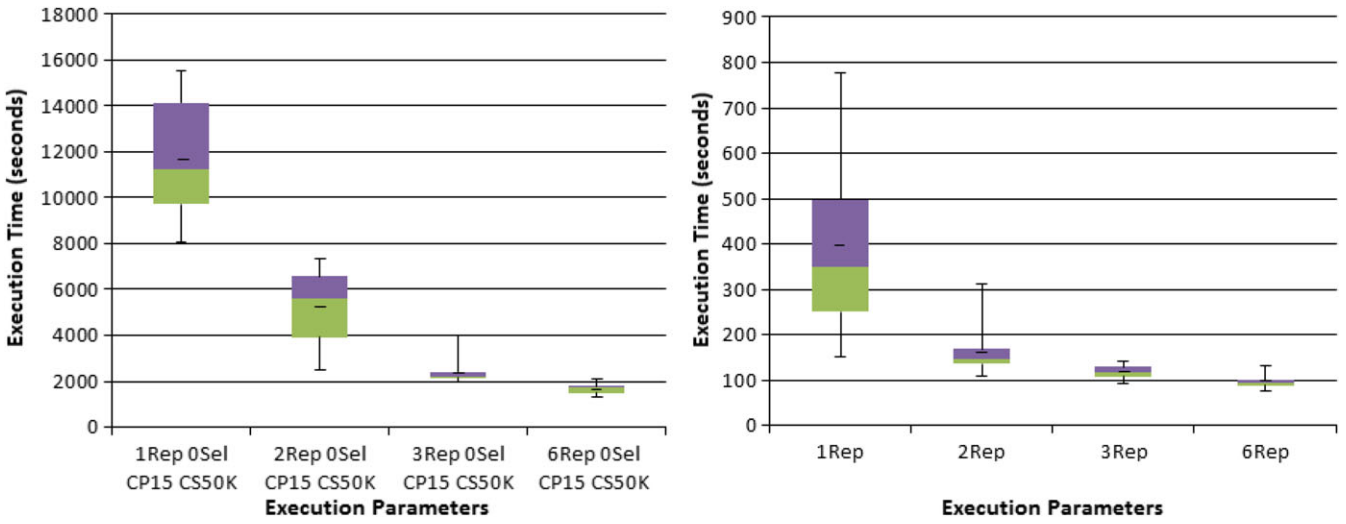

FIGURE 9 Execution times for REMD (left) and SoE (right) with different levels of replication. There is no host selection. The checkpoint interval is 15 minutes, and the checkpoint size is 50 kB for REMD. No checkpointing is employed for SoE

It is clear from the figure that employing replication has a significant positive impact. This is not surprising as application progress in Volpex is dictated by the fastest replica of each process. Employing 2 replicas dramatically improves the execution time mean, median, and variance for both the codes, in comparison to no replication. Further increasing the degree of replication to 3 and 6 offers smaller improvements. The impact on both codes is broadly similar implying that the impact of replication is generally independent of the application characteristics. Replication in Volpex is implemented such that the application progress is aligned with the fastest live replica of each process. The results show that replication is able to effectively mask the detrimental impact of failures, as well as slow processes. We note that adding replicas typically reduces variability in execution times significantly, even when the improvement in median times is modest. The reason is that replication has the most impact in the presence of failures and slow nodes and the number of instances of these can vary considerably between runs. We also observe that a much larger fraction of possible improvement with replication is achieved with just 2 replicas for SoE, which we attribute to the lower execution time. The fact that performance continues to improve up to 6 replicas also shows that the Dataspace server is not a bottleneck even as fairly large number of process instances (approximately 400 for 6 replicas) communicate through the Dataspace server.

Impact of host selection Figure 10 presents experimental results for REMD and SoE executed with and without host selection. The impact of host selection is evaluated in scenarios with and without replication. For REMD, fixed checkpoint interval of 15 minutes and checkpoint size of 50 kB were used. No checkpointing was used for SoE.

The figure illustrates that host selection improves performance for both codes, with and without replication. The improvement in mean and median execution time is very modest for SoE with replication. The reason is simply that replication alone already provides close to optimal performance. Figure 10 can also be used to compare the relative impact of adding replication versus adding host selection. It is clear that replication has more positive impact than host selection, especially in the variability of execution time. The reason is that host selection is a mere prediction which will often prove to be incorrect as even "good" hosts fail. For host selection, we employ a basic predictor called *Lastval*, which is simply based on the availability of the host in the immediate past. If a host was available in the recent past, there is a good chance that it would be available in the near future. It is possible that some "good hosts" with good hardware configuration and high upload/download bandwidth may be blacklisted if these computers become idle and active repeatedly during node recruitment. If CPU utilization of such computers fluctuates a lot, they will be frequently recruited and removed from the active node pool. Every time there is a failure of a task due to removal of the host, that host will not be allowed to participate in computations for a probation period, and repeated task failures can ultimately lead to the host being blacklisted. Hence, in some runs, host selection may have no or minimum benefit, while replication provides "backup" execution more consistently. When host selection and replication are employed at the same time, the performance is consistently good for both applications as host selection minimizes the usage of failure prone hosts, while the impact of any failures that still happen is minimized by replication. In our evaluation experiments, approximately 33% of hosts were rejected in the process of host selection, but we should point out that the rejected hosts can be employed for other applications. The performance with host selection and replication is similar to performance with higher degrees of replication that was shown in Figure 9, while being more resource efficient.

Impact of Checkpointing Checkpointing is an integral component of execution of long running codes under Volpex, although it is typically not sufficient for efficient execution. Management of replication also relies on checkpointing to create new replicas dynamically. Checkpointing interval is an important execution parameter. A considerable body of research addresses determining the optimal checkpointing interval for ordinary

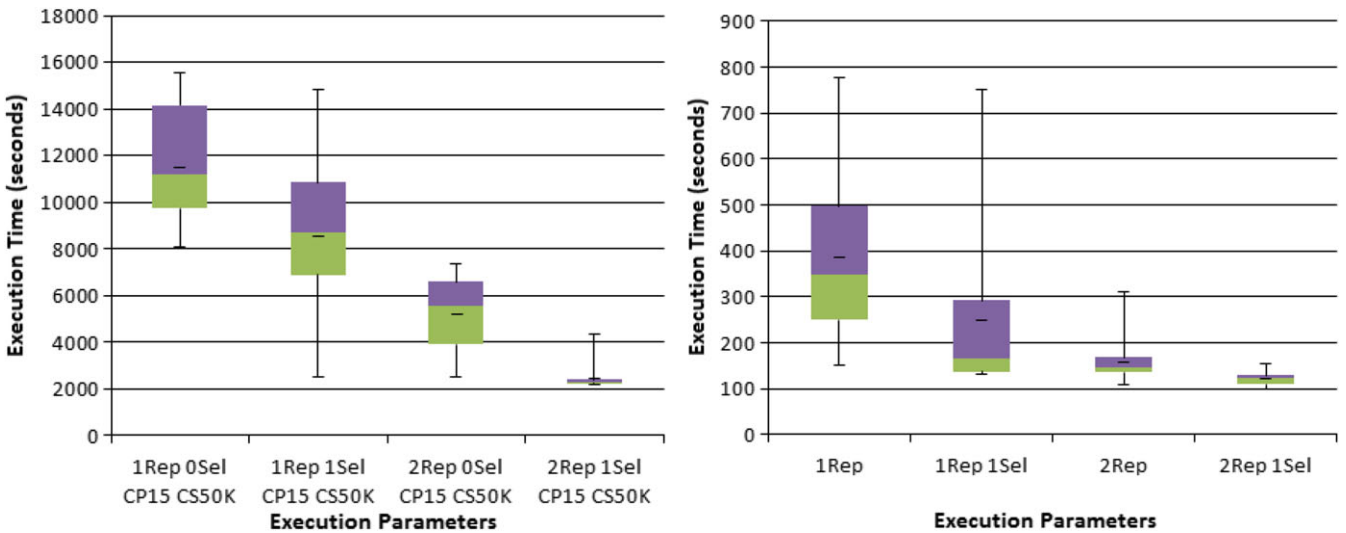

FIGURE 10 Execution times for REMD (left) and SoE (right) with and without host selection for replication levels of 1 and 2. The checkpoint interval is 15 minutes and checkpoint size is 50 kB for REMD. No checkpointing is employed for SoE

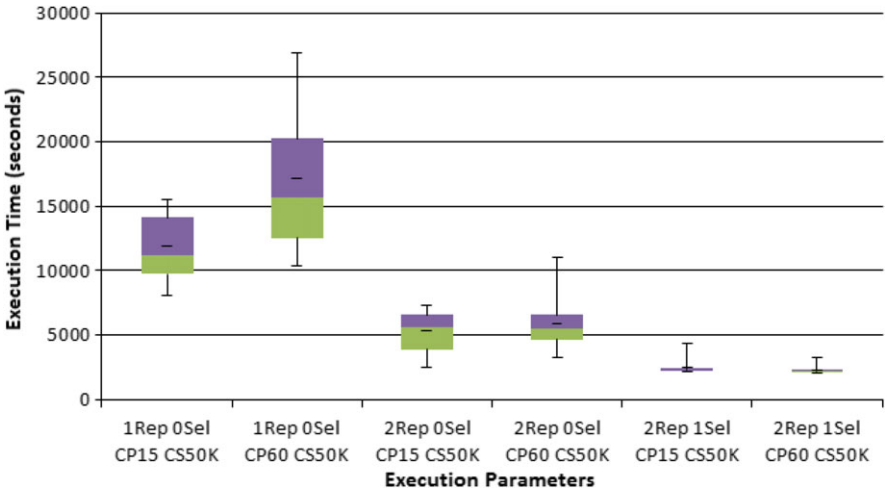

FIGURE 11 Execution time for REMD with short and long checkpoint intervals

processes[26,27] and some of the challenges in volunteer environments have been addressed.[28] However, we are not aware of any work that addresses optimization of the checkpointing interval for communicating parallel codes on heterogeneous volunteer nodes when a combination of checkpointing and replication is employed. Results presented thus far in the paper were for a checkpoint interval of 15 minutes and a checkpoint size of 50 kB. The checkpoint interval of 15 minutes was selected empirically as a good choice for the type of scenarios employed for evaluation; reducing the checkpoint interval led to a slow and gradual increase in execution times. A checkpoint size of 50 kB is sufficient for REMD and other codes that essentially exchange state information; a lower checkpoint size had little impact on execution time in our experiments. Details of these results are omitted for brevity. In this section we develop more insight into the role of checkpointing in Volpex. Results are presented to see the performance impact of considerably lowering the checkpointing overhead by increasing the checkpoint interval to 60 minutes, and the performance impact of large checkpoint sizes with a simulated checkpoint size of 5 MB. Results are presented only for the REMD code as the SoE code is not a suitable candidate for checkpointing because of a relatively short execution time.

Figure 11 presents experimental results for REMD executed with a checkpoint interval of 60 minutes along with earlier results for a checkpoint interval of 15 minutes. The checkpoint size is small, and the execution scenarios are as follows: no replication or host selection, replication (degree 2) but no host selection, and replication and host selection.

We observe that increasing the checkpoint interval to 60 minutes has significant detrimental impact on execution without replication or host selection. The reason is that a single host failure can lead to a long delay and we certainly expect many failures in an execution. Clearly, a well chosen checkpoint interval is very important in a volunteer environment, especially if good node selection and replication is not available. Figure 11 also shows that the impact of checkpoint interval on execution time is minimal when replication is also employed.

One way to interpret these results is that replication allows us to have a longer checkpointing interval with no negative impact. In this experiment, the size of the checkpoints is relatively small (50 kB). An application with large footprint checkpoints could benefit more with a reduced frequency of checkpoints made possible by host selection and replication. In the following experiments, we will examine execution behavior with larger checkpoints.

Figure 12 presents execution time for REMD code with replication and with and without host selection, for synthetically increased checkpoint size of 5 MB, a 100 fold increase over the original checkpoint size of 50 kB. The results show a significant increase in the execution time for the case where only replication is employed, and a very slight increase when replication and checkpointing is employed. When the checkpoint size is large, the overhead for creating and uploading the checkpoint to the server becomes significant, which explains the increase in execution time. However, the difference in execution time when both replication and host selection are employed is low as faster execution times imply fewer instances of checkpointing. Also, host selection filters out hosts with slow upload speed which is an important factor for large checkpoints.

Figure 13 shows results from REMD experiments with the same configurations (replication degree of 2 and with or without host selection) as Figure 12 but for a checkpoint size of 5 MB and with checkpoint interval of 15 and 60 minutes. We observe that the execution time actually decreases for the case of replication and no host selection, as the checkpoint interval is increased from 15 to 60 minutes. The explanation is that, when the checkpoint size is large, the overhead for creating and uploading the checkpoint to the server becomes significant and exceeds the benefits from more frequent checkpoints. The difference in execution time when both replication and host selection are employed is minimal as the overhead of checkpointing and recovery was low because of reduced number of failures and faster execution, and because host selection filters out hosts with slow upload speed. With the help of replication and host selection to mitigate the impact of failures, an application with a large checkpoint size can benefit from using a larger interval between checkpoints.

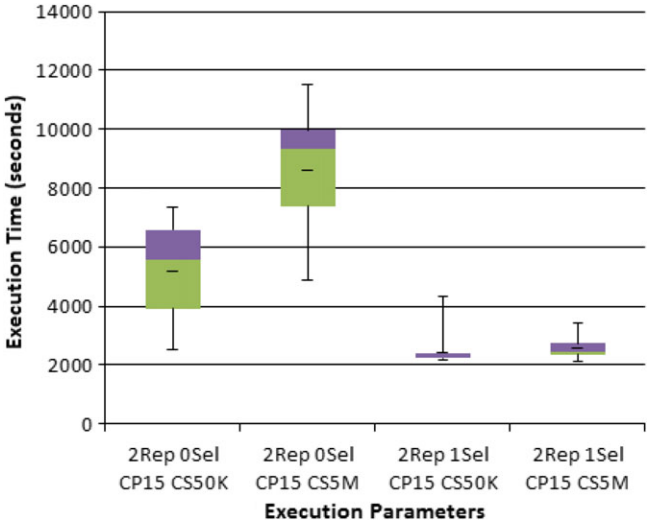

FIGURE 12 Execution time for REMD with small and large checkpoint sizes

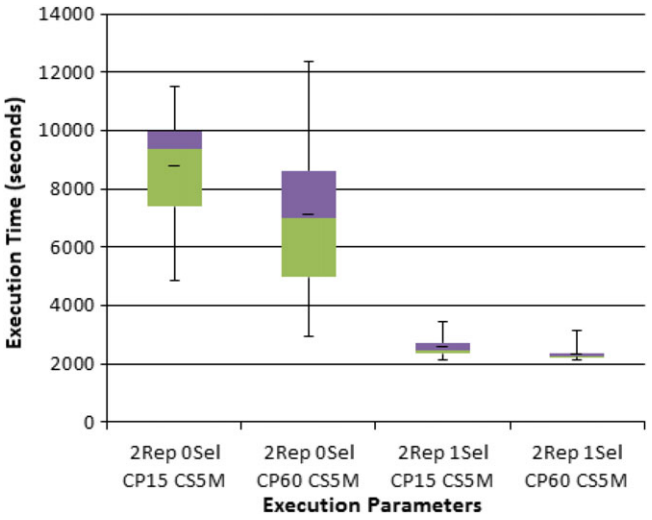

FIGURE 13 Execution time for REMD with large checkpoint size and varying checkpoint intervals

6.4 | Summary of results

Results presented in Section 6.2 demonstrate that the implementation of Volpex API is an effective and efficient way to communicate between replicated processes in a distributed volunteer environment. Results presented in Section 6.3 analyze the impact of replication, host selection, and checkpointing on execution of parallel applications with REMD code as the driving example. Performance in various execution scenarios discussed is also plotted in Figure 14. The experimental results clearly demonstrate that effective and predictable execution of parallel jobs in a volunteer environment is possible only with some combination of host selection, replication, and checkpointing. These methods often work more effectively in combination than individually. Some of the key conclusions are as follows.

1. Replication is an effective approach toward efficient execution. Adding a set of replica processes can lead to more than halving of execution time, implying that it can be very cost effective. Also, all scenarios where an execution time close to optimal was achieved employ some degree of replication.
2. Checkpointing is a key component of execution on volunteer nodes, although checkpointing alone is not sufficient for effecive execution.
3. Having 2 replicas per process along with host selection and checkpointing at modest time intervals is likely to be the most cost effective strategy in many execution scenarios.

While there are many other types of applications and execution scenarios, we believe the results presented provide significant insight into the value of various approaches to managing execution in a volunteer execution environment. Additional results are available in the work of Nguyen.[10]

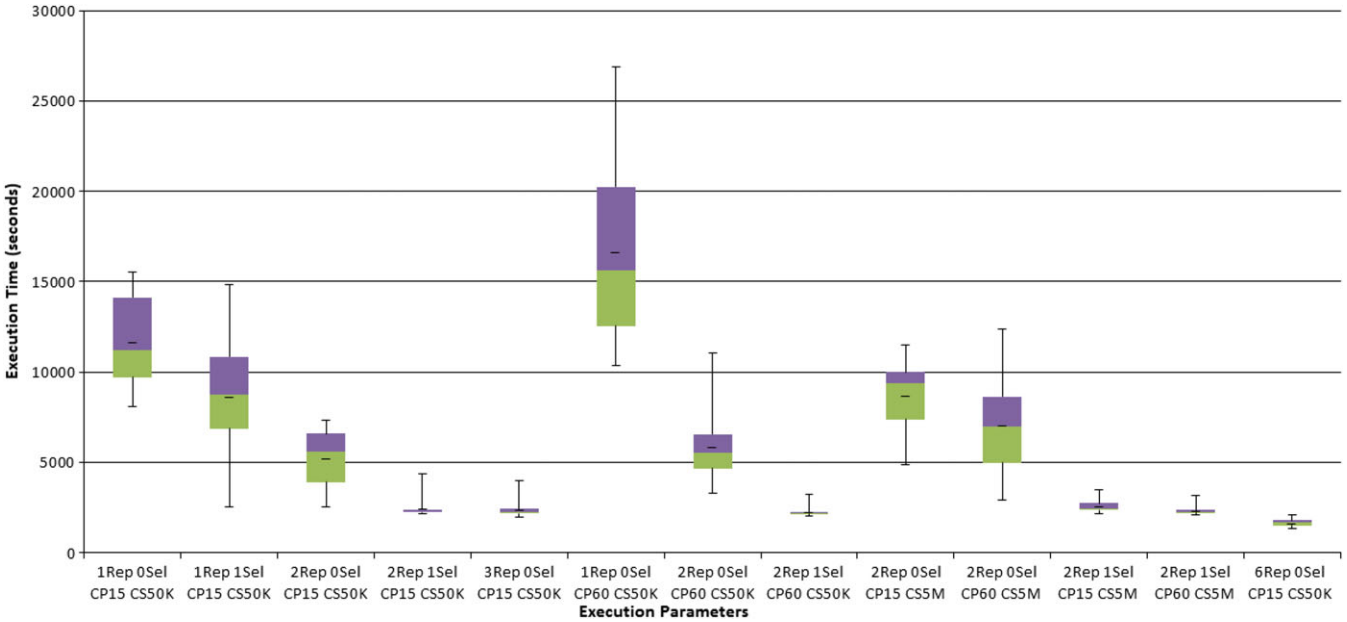

FIGURE 14 All results for different REMD execution scenarios

We also note that different methods have different costs; replication doubles the resources that are used, node selection limits the number of nodes that are useful, and the checkpointing cost is application specific. The cost and availability of resources is also a key factor in execution management.

7 | RELATED WORK

Idle desktops are widely used for parallel and distributed computing. There has been a lot of research toward improving its performance and productivity.[29-31] The Berkeley Open Infrastructure for Network Computing (BOINC)[4] is a middleware system widely used for volunteer computing. Condor,[32] is a workload management system that can effectively harness wasted CPU power. An interesting approach, where volunteer computing middleware is integrated in the Web browser, is explored in Weevilscout.[5] Other systems that build desktop computing grids include Entropia,[33] iShare[34] and Comcute.[35] Mechanisms applied for fault tolerance in PC grids, such as redundancy in BOINC and checkpointing in Condor, are important for long running sequential and bag-of-tasks codes, but are generally not sufficient for communicating parallel programs.

Linda,[14] TSpaces,[36] JavaSpaces,[37] and many others represent a *Put/Get* model of coordination and communication among parallel processes based on a logically global associative memory. The Volpex Dataspace API,[7,8] the underlying framework for this research, is also based on Put/Get calls to a logically shared memory, the key enhancement being efficient support for distributed replicated processes.

The notion of having a distributed shared memory programming model using an abstract data space has also been explored in the work of Docan et al,[38] without however, support for fault tolerance.

Several implementations of the MPI specification have focused on fault-tolerance mechanisms. The vast majority of these projects rely on system level check-pointing and automatic roll back of the application, often relying on a third party system level library such as BLCR.[21] Representatives of these libraries include LAM/MPI,[23] Open MPI,[39] and MPICH-V.[22] Some approaches have utilized process replication to create robust MPI libraries, such as MPI/FT,[40] P2P-MPI,[41] rMPI,[42] and VolpexMPI,[9] the latter being developed as a complementary aspect of the work presented in this paper. P2P-MPI and VolpexMPI are the only libraries targeting volunteer computing within this group. Process replication is also deployed in the MOON framework[43] for MapReduce applications on volunteer resources.

The key innovation of this work is customizing and integrating replication and checkpointing to build an execution framework that can handle very high failure rates and other peculiarities of volunteer environments. A *Put/Get* model of communication is deployed as it is particularly suitable for a distributed environment. The runtime system will be adapted for an MPI implementation in the future.

An important component of the Volpex execution environment is a host selection module that leverages ample existing research. Multiple predictors of future availability of a host based on previous history are described and evaluated in the works of Andrzejak et al[18] and Rood and Lewis.[19] Prediction of application execution time in a volunteer environment is studied in the works of Estrada et al[44] and Taufer et al.[45] In contrast to research discussed above, this paper focuses on communicating parallel jobs rather than "bag-of-tasks" jobs, and on actual execution on a testbed rather than simulation. The predictor we employ is primarily based on recent host availability, a choice made partly because the aforementioned author[18] showed that this simple predictor was nearly as effective as other complex and sophisticated predictors.

Checkpointing and replication are important components of the Volpex execution environment. Zheng and Subhlok[6] proposed simple models for characterizing the impact of parameters on the efficiency of checkpointing with restart and replication schemes. Cirne et al[46] gave a deep evaluation of replication in large scale distributed system with replication scheduler. Uncoordinated checkpointing[22] is the only viable option in a volunteer environment as coordinated checkpointing[23] requires finer synchronization and is difficult to handle with redundancy. Volpex currently employs application level checkpointing in the context of BOINC.[20] Our approach to sharing of checkpoints between replicas is partially motivated by Domingues et al.[47]

In recent years, significant efforts have also been invested in the robust execution of Big Data applications, with two major areas overlapping with the volunteer computing community: failures resilience and replication. Hadoop[48] as the most prominent representative of the Big Data software uses block-level replication to achieve robustness on the file system level.[49] This allows to re-execute a map or reduce instance in case a node becomes unavailable. To a lesser extent, the MapReduce framework also deploys process level replication for performance improvement, typically in the final stages of the mapper step. CrowdProcess[50] is a commercial product that employs volunteer computing to solve scientific problems on a Web based distributed computing platform with the MapReduce framework. HybridMR[51] is another approach that combines desktop grid and cloud infrastructures for MapReduce computation on hybrid computing environment.

8 | CONCLUSIONS

This paper presented a framework for "free" execution of communicating parallel programs on volunteer nodes based on the concept of autonomous redundant processes. This paper introduces the Volpex Dataspace API that allows efficient Put/Get operations on an abstract global shared memory. An innovative communication model and implementation ensure consistent execution results in the presence of multiple asynchronous invocations of a single communication call due to redundancy for fault tolerance.

A key contribution of this paper is an execution environment that is able to effectively convert a volunteer PC grid into a virtual cluster for the execution of communicating parallel applications. Replication, checkpointing, and host selection are combined and integrated in new ways to meet the daunting challenge of effective parallel execution on failure prone and heterogeneous nodes with dynamically changing execution behavior. Key innovations include fully autonomous replicas, unified replication and checkpointing, co-operative checkpointing across process replicas, hot spares for quick restarts, termination and restart of lagging processes, and high level user control over resource and performance trade-offs.

This work provides the first implementation and evaluation of the combination of these methods for volunteer computing with a real-life testbed. The system has been integrated with BOINC middleware to provide, to the best of our knowledge, the first comprehensive solution to the reliable execution of general parallel programs on volunteer nodes. Experimental results show that the framework can execute parallel codes effectively and efficiently execute communicating parallel codes.

Ongoing research is addressing automation of selection of runtime methods, such as the optimal combination of replication and checkpointing best suited to a given application. Research is also addressing seamless execution in the presence of Byzantine faults that cause reporting of incorrect results instead of failures. The purpose is to build a robust, elegant and practical environment for parallel computing on volunteer nodes. At the same time, the principles and methods developed in this work are applicable to other heterogeneous environments where state of the art for fault tolerance is inadequate, such as large clusters and computation clouds.

ACKNOWLEDGMENTS

Partial support for this work was provided by the National Science Foundation under award CNS-0834750, MCB-0919974, and CRI-0958464. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the National Science Foundation. We would like to acknowledge the contributions of Keith Crabb at UH Research Computing Center and Tsung-I "Mark" Huang at Texas Learning and Computation Center toward providing the infrastructure for this project. Margaret Cheung and Qian Wang supported the development of REMD, the driving application in this project. Nagarajan Kanna, and Eshwar Pedamallu developed the earlier versions of the Volpex framework employed in this research.

ORCID

Mohammad Tanvir Rahman http://orcid.org/0000-0002-5603-7043

REFERENCES

1. Globus Toolkit. http://toolkit.globus.org/toolkit/
2. UNICORE. https://www.unicore.eu/

3. Benedyczak K, Schuller B, Sayed MPE, Rybicki J, Grunzke R. UNICORE 7—Middleware services for distributed and federated computing. Paper presented at: 14th International Conference on High Performance Computing Simulation (HPCS); 2016; Innsbruck, Austria. https://doi.org/10.1109/HPCSim.2016.7568392
4. Anderson DP. BOINC: A system for public-resource computing and storage. Paper presented at: 5th IEEE/ACM International Workshop on Grid Computing. IEEE Computer Society; 2004; Washington, DC. https://doi.org/10.1109/GRID.2004.14
5. Cushing R, Putra GHH, Koulouzis S, Belloum A, Bubak M, de Laat C. Distributed computing on an ensemble of browsers. *IEEE Internet Comput*. 2013;17(5):54-61.
6. Zheng R, Subhlok J. A quantitative comparison of checkpoint with restart and replication in volatile environments [Technical Report] UH-CS-08-06. University of Houston; 2008.
7. Kanna N, Subhlok J, Gabriel E, Rohit E, Anderson D. A communication framework for fault-tolerant parallel execution. Paper presented at: 22nd International Conference on Languages and Compilers for Parallel Computing (LCPC). Springer-Verlag; 2010; Berlin, Heidelberg.
8. Rohit E, Nguyen H, Kanna N, et al. A robust communication framework for parallel execution on volunteer PC grids. Paper presented at: 11th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGrid); 2011; Newport Beach, CA. https://doi.org/10.1109/CCGrid.2011.72
9. Anand R, LeBlanc T, Gabriel E, Subhlok J. A robust and efficient message passing library for volunteer computing environments. *J Grid Comput*. 2011;9(3):325-344.
10. Nguyen H. *An Execution Environment for Robust Parallel Computing on Volunteer PC Grids* [Master's thesis]. Houston, TX: University of Houston; 2011.
11. Nguyen H, Pedamallu E, Subhlok J, et al. An execution environment for robust parallel computing on volunteer PC grids. Paper presented at: 41st International Conference on Parallel Processing (ICPP); 2012; Pittsburgh, PA.
12. Islam MT, Nguyen H, Subhlok J, Gabriel E. Efficient message logging to support process replicas in a volunteer computing environment. Paper presented at: 29th IEEE International Parallel and Distributed Processing Symposium Workshop (IPDPSW); 2015; Hyderabad, India. https://doi.org/10.1109/IPDPSW.2015.91
13. Rahman MT, Nguyen H, Subhlok J, Pandurangan G. Checkpointing to minimize completion time for inter-dependent parallel processes on volunteer grids. Paper presented at: 16th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGrid); 2016; Cartagena, Colombia. https://doi.org/10.1109/CCGrid.2016.78
14. Carriero N, Gelernter D. The S/Net's Linda kernel. *ACM Trans Comput Syst*. 1986;4(2):110-129.
15. Wang Q, Liang KC, Czader A, Waxham MN, Cheung MS. The effect of macromolecular crowding ionic strength and calcium binding on calmodulin dynamics. *PLoS Comput Biol*. 2011;7(7):1-16.
16. Kondo D, Andrzejak A, Anderson DP. On correlated availability in internet-distributed systems. Paper presented at: 9th IEEE/ACM International Conference on Grid Computing (Grid 2008); 2008; Tsukuba, Japan.
17. Javadi B, Kondo D, Vincent JM, Anderson DP. Discovering statistical models of availability in large distributed systems: an empirical study of SETI@home. *IEEE Trans Parallel Distrib Syst*. 2011;22(11):1896-1903.
18. Andrzejak A, Kondo D, Anderson D. Exploiting non-dedicated resources for cloud computing. Paper presented at: 12th IEEE/IFIP Network Operations and Management Symposium (NOMS); 2010; Osaka, Japan.
19. Rood B, Lewis MJ. Availability prediction based replication strategies for grid environments. Paper presented at: 10th IEEE/ACM International Conference on Cluster, Cloud and Grid Computing (CCGrid); 2010; Melbourne, VIC, Australia.
20. Anderson DP, Christensen C, Allen B. Designing a runtime system for volunteer computing. Paper presented at: 19th ACM/IEEE Conference on Supercomputing(SC). ACM; 2006; Tampa, FL.
21. Duell J, Hargrove P, Roman E. The Design and Implementation of Berkeley Lab's Linux Checkpoint/Restart. Berkeley Lab Technical Report (publication LBNL-54941); 2002.
22. Bouteiller B, Cappello F, Herault T, Krawezik K, Lemarinier P, Magniette M. MPICH-v2: A fault tolerant MPI for volatile nodes based on pessimistic sender based message logging. Paper presented at: 16th ACM/IEEE Conference on Supercomputing(SC); 2003; Phoenix, AZ. https://doi.org/10.1145/1048935.1050176
23. Sankaran S, Squyres JM, Barrett B, et al. The LAM/MPI checkpoint/restart framework: system-initiated checkpointing. *Int J High Perform Comput Appl*. 2005;19(4):479-493.
24. Sugita Y, Okamoto Y. Replica-exchange molecular dynamics method for protein folding. *Chem Phys Lett*. 1999;314:141-151.
25. Case D, Pearlman DA, Caldwell JW, et al. Amber 6 Manual; 1999.
26. Daly JT. A higher order estimate of the optimum checkpoint interval for restart dumps. *Futur Gener Comput Syst*. 2006;22(3):303-312.
27. Young JW. A first order approximation to the optimum checkpoint interval. *Commun ACM*. 1974;17(9):530-531.
28. Bouguerra MS, Kondo D, Trystram D. On the scheduling of checkpoints in desktop grids. Paper presented at: 11th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing; 2011; Newport Beach, CA.
29. Kondo D. Preface to the special issue on volunteer computing and desktop grids. *J Grid Comput*. 2009;7(4):417-418.
30. Toth D, Finkel D. Improving the productivity of volunteer computing by using the most effective task retrieval policies. *J Grid Comput*. 2009;7(4):519-535.
31. Heien EM, Anderson DP, Hagihara K. Computing low latency batches with unreliable workers in volunteer computing environments. *J Grid Comput*. 2009;7(4):501-518.
32. Thain D, Tannenbaum T, Livny M. Distributed computing in practice: the Condor experience. *Concurrency Computat Pract Exper*. 2005;17(2-4):323-356.
33. Kondo D, Taufer M, Brooks CL, Casanova H, Chien AA. Characterizing and evaluating desktop grids: An empirical study. Paper presented at: 18th International Parallel and Distributed Processing symposium (IPDPS); April 2004; Santa Fe, NM.
34. Ren X, Eigenmann R. iShare—Open internet sharing built on peer-to-peer and web. Paper presented at: European Grid Conference, EGC'05; 2005; Amsterdam, Netherlands.

35. Czarnul P, Kuchta J, Matuszek M. *Parallel Computations in the Volunteer–Based Comcute System*. Berlin Germany: Springer-Verlag Berlin Heidelberg; 2014;261-271.
36. TSpaces. http://www.almaden.ibm.com/cs/tspaces/
37. Noble MS, Zlateva S. Scientific computation with JavaSpaces. Paper presented at: 9th International Conference on High-Performance Computing and Networking (HPCN Europe). Springer-Verlag Berlin Heidelberg; 2001:657-666; Berlin, Germany.
38. Docan C, Parashar M, Klasky S. DataSpaces: an introduction and coordination framework for coupled simulation workflows. Paper presented at: 19th ACM International Symposium on High Performance Distributed Computing (HPDC). 2010:25-36; Chicago, Illinois.
39. Hursey J, Mattox TI, Lumsdaine A. Interconnect agnostic checkpoint/restart in Open MPI. Paper presented at: 18th ACM International Symposium on High Performance Distributed Computing (HPDC). ACM; 2009:49-58; New York, NY.
40. Batchu R, Neelamegam JP, Cui Z, Beddhu M, Skjellum A, Dandass Y. MPI/FT: Architecture and taxonomies for fault-tolerant, message-passing middleware for performance-portable parallel computing. Paper presented at: 1st IEEE International Symposium of Cluster Computing and the Grid; 2001; Brisbane, Queensland, Australia.
41. Genaud S, Rattanapoka C. Large-scale experiment of co-allocation strategies for peer-to-peer supercomputing in P2P-MPI. Paper presented at: 22nd IEEE International Symposium on Parallel and Distributed Processing (IPDPS); 2008; Miami, FL.
42. Stearley JR, Riesen RE, Laros JH, et al. Increasing fault resiliency in a message-passing environment; 2009.
43. Lin H, Ma X, Archuleta J, Feng WC, Gardner M, Zhang Z. MOON: Mapreduce On Opportunistic eNvironments. Paper presented at: 19th ACM International Symposium on High Performance Distributed Computing (HPDC). ACM; 2010:95-106; New York, NY, USA.
44. Estrada T, Taufer M, Anderson DP. Performance prediction and analysis of BOINC projects: An empirical study with emBOINC. *J Grid Comput*. 2009;7(4):537-554.
45. Taufer M, Kerstens A, Estrada T, Flores D, Teller PJ. SimBA: A discrete event simulator for performance prediction of volunteer computing projects. Paper presented at: 21st International Workshop on Principles of Advanced and Distributed Simulation (PADS'07); June 2007; San Diego, CA.
46. Cirne W, Brasileiro FV, da Silva DP, Góes LFW, Voorsluys W. On the efficacy, efficiency and emergent behavior of task replication in large distributed systems. *Parallel Comput*. 2007;33:213-234.
47. Domingues P, Silva JG, Silva L. Sharing checkpoints to improve turnaround time in desktop grid computing. Paper presented at: 20th International Conference on Advanced Information Networking and Applications (AINA); 2006; Vienna, Austria.
48. White T. *Hadoop: The Definitive Guide*. 2nd ed. Sebastopol, CA: O'ReillyMedia Inc.; 2010.
49. Shafer J. The Hadoop distributed filesystem: Balancing portability and performance. Paper presented at: 11th IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS); 2010; White Plains, NY.
50. CrowdProcess. https://crowdprocess.com/
51. Tang B, He H, Fedak G. HybridMR: a new approach for hybrid MapReduce combining desktop grid and cloud infrastructures. *Concurrency Computat Pract Exper*. 2015;27(16):4140-4155.

How to cite this article: Subhlok J, Nguyen H, Gabriel E, Rahman MT. Resilient parallel computing on volunteer PC grids. *Concurrency Computat Pract Exper*. 2018;e4478. https://doi.org/10.1002/cpe.4478